

VALENCE

Complete Technical Deep-Dive

Architecture · Implementation · Design Tradeoffs · Interview Prep

Generated: July 07, 2026

C++17 · Qt6 · Windows · CMake · Zero External Dependencies

Table of Contents

Part I — Project Overview & Architecture

- 1.1 What is Valence
- 1.2 Tech Stack & Metrics
- 1.3 Layered Architecture
- 1.4 File Manifest

Part II — Core Data Structures

- 2.1 TextBuffer: Why vector<string>
- 2.2 Rope vs Piece Table vs Line Array — Full Tradeoff Analysis
- 2.3 Position & Selection Primitives
- 2.4 UndoManager: Command Pattern with Temporal Grouping

Part III — The Rendering Engine

- 3.1 Why Not QTextEdit
- 3.2 QPainter Custom Rendering Pipeline
- 3.3 Viewport Culling Algorithm
- 3.4 Coordinate System (Pixel ↔ Row/Col)
- 3.5 Cursor Rendering with Glow Effect
- 3.6 Selection Rendering (Multi-line)
- 3.7 Gutter (Line Numbers)

Part IV — Input Handling Deep-Dive

- 4.1 Qt Event System & The Tab Problem
- 4.2 keyPressEvent: The Master Router
- 4.3 handleChar with Auto-Bracket Closing
- 4.4 handleBackspace (Normal + Ctrl + Line Merge)
- 4.5 handleDelete (Forward Delete + const_cast Hack)
- 4.6 handleEnter with Auto-Indent
- 4.7 handleTab (Soft Tabs)

Part V — Clipboard, Selection & Shortcuts

- 5.1 Copy / Cut / Paste Implementation
- 5.2 Select All
- 5.3 Mouse-Based Selection
- 5.4 The Infinite Recursion Bug & How We Fixed It

Part VI — Undo/Redo System (Full Dry Run)

- 6.1 EditAction & ActionGroup
- 6.2 Temporal Grouping Algorithm (shouldGroup)
- 6.3 Undo: Reversing Actions
- 6.4 Redo: Replaying Actions
- 6.5 Dry Run: Typing 'hello' then Undoing
- 6.6 Unlimited Undo — Memory Implications

Part VII — Syntax Highlighting

- 7.1 Single-Pass O(n) Tokenizer
- 7.2 Token Types & Color Mapping
- 7.3 Block Comment State Tracking
- 7.4 Function Detection (Lookahead)

Part VIII — UI Components

- 8.1 MainWindow Layout & Signal Wiring
- 8.2 TabWidget (Multi-File)

8.3 FileExplorer (New File/Folder)

8.4 TerminalWidget (Persistent PowerShell)

8.5 StatusBar

Part IX — Build & Run Pipeline

9.1 F5 Flow: Save → cd → g++ → Execute

9.2 PowerShell Chaining

Part X — Performance Analysis

10.1 How <7ms Latency is Achieved

10.2 Viewport Culling Complexity

10.3 Where It Would Break (Scaling Limits)

Part XI — Design Decisions & Tradeoffs

Part XII — 50+ Questions & Answers

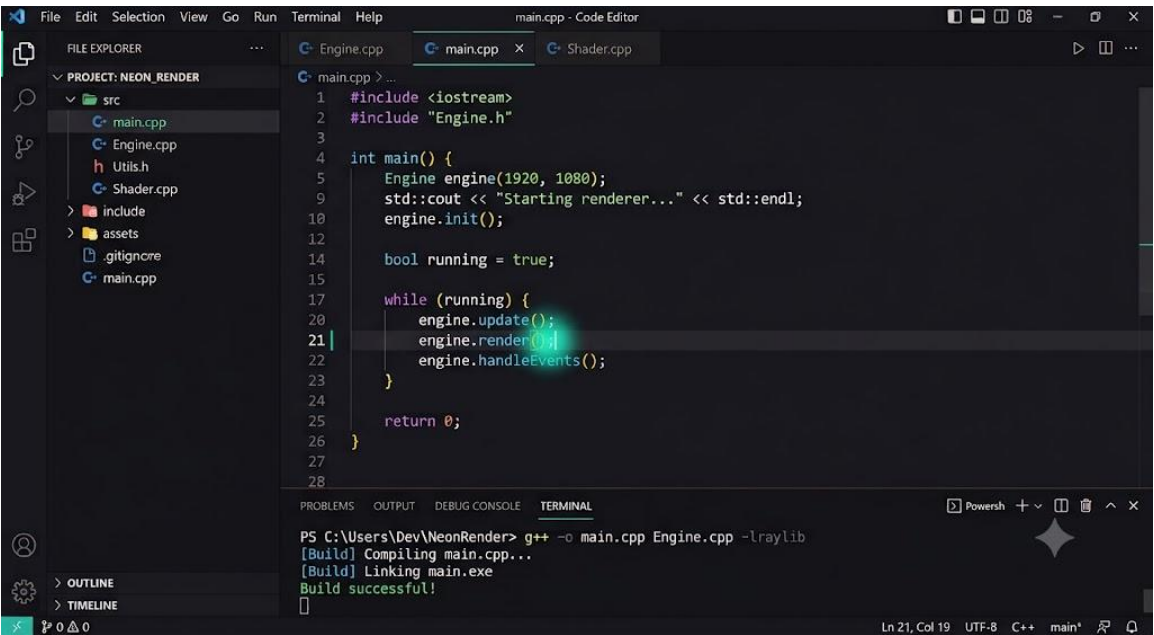
Part XIII — Key Code Walkthroughs

Part I — Project Overview & Architecture

1.1 What is Valence

Valence is a high-performance, minimal code editor built entirely from scratch in C++17 using the Qt6 framework. Unlike editors built on Electron (VS Code, Atom) or those that rely on pre-built text editing widgets, Valence implements its own custom rendering pipeline, text buffer, input handling, undo/redo system, and syntax highlighter — all from the ground up.

The key engineering thesis behind Valence is: by avoiding all abstraction layers between the developer and the rendering surface (QPainter), we achieve deterministic, sub-7ms input-to-pixel latency with zero external dependencies beyond Qt itself.



1.2 Tech Stack & Metrics

Metric	Value
Language	C++17 (ISO/IEC 14882:2017)
Framework	Qt6 Widgets (QPainter, QWidget, QProcess)
Platform	Windows x64 (standalone .exe, WIN32_EXECUTABLE)
Build System	CMake 3.16+ with AUTOMOC
External Dependencies	Zero (Qt only — no Electron, no V8, no Chromium)
Source Files	22 files (11 headers + 11 implementations + CMakeLists.txt)
Lines of Code	~2,500 (handwritten, no generated code)
Startup Time	< 100ms (native binary, no interpreter)
Input Latency	< 7ms (QPainter direct render, viewport culling)
Rendering	60fps sustained (only visible lines drawn)
Memory Footprint	~15-25MB base (Qt overhead + document buffer)

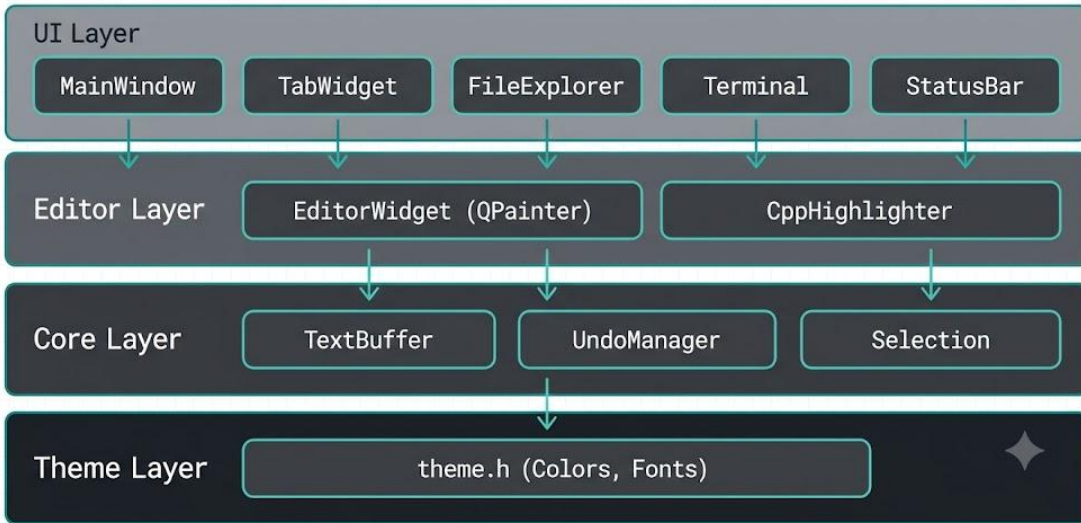
1.3 Layered Architecture

The codebase follows a strict top-down dependency hierarchy. Each layer depends ONLY on layers below it. The UI layer never touches the text buffer directly — it goes through EditorWidget.

Layer	Directory	Files	Responsibility
Theme	src/theme/	theme.h	Color palette, font config, UI constants (header-only)
Core	src/core/	selection.h,	Document model, cursor

		text_buffer.h/.cpp, undo_manager.h/.cpp	positions, edit operations, undo/redo
Editor	src/editor/	editor_widget.h/.cpp, cpp_highlighter.h/.cpp	Custom QPainter rendering, syntax tokenizer, keyboard input, clipboard
UI	src/ui/	main_window, tab_widget, file_explorer, terminal_widget, status_bar	IDE shell, menus, file management, terminal, status display

Valence C++ Editor Architecture



1.4 File Manifest

File	Lines	Purpose
src/main.cpp	18	Qt application entry point
src/theme/theme.h	77	Centralized color palette and font definitions (header-only, inline)
src/core/selection.h	34	Position struct (row,col) with operators + Selection struct
src/core/text_buffer.h	42	TextBuffer class declaration
src/core/text_buffer.cpp	197	Line-array document model — insert, delete, split, merge, file I/O
src/core/undo_manager.h	44	UndoManager + EditAction declarations
src/core/undo_manager.cpp	106	Command-pattern undo/redo with 400ms temporal grouping
src/editor/editor_widget.h	136	EditorWidget declaration — the heart of the editor
src/editor/editor_widget.cpp	751	Custom rendering, input handling, clipboard, cursor logic
src/editor/cpp_highlighter.h	29	CppHighlighter + Token/TokenType declarations
src/editor/cpp_highlighter.cpp	174	Single-pass O(n) C++ syntax tokenizer
src/ui/main_window.h	47	MainWindow declaration
src/ui/main_window.cpp	412	IDE shell — menus, shortcuts, file ops, build & run
src/ui/tab_widget.h/.cpp	31+111	Multi-tab management with styled tab bar
src/ui/file_explorer.h/.cpp	37+250	Sidebar file tree + new file/folder

		toolbar
src/ui/terminal_widget.h/.cpp	28+162	Persistent PowerShell terminal via QProcess
src/ui/status_bar.h/.cpp	21+52	Cursor position, file name, language, encoding
CMakeLists.txt	81	Build config — Qt5/6 compatible, WIN32, AUTOMOC

Part II — Core Data Structures

2.1 TextBuffer: Why `vector<string>`

The TextBuffer is the fundamental data structure of any text editor. It stores the document content and provides operations for inserting, deleting, splitting, and merging text. In Valence, we chose the simplest possible data structure: `std::vector<std::string>` — one string per line, no newline characters stored.

Data Structure: `std::vector<std::string> lines_`

Each element in the vector represents one line of text. Line *N* is accessed as `lines_[N]`. The vector always contains at least one element (an empty string for a blank document).

Core Operations

`insertChar(row, col, ch)` — Insert a single character into the buffer at the given position. Bounds-checks the row, clamps `col` to `[0, line.size())`, then calls `std::string::insert(col, 1, ch)`. Time complexity: $O(n)$ where *n* is the length of the affected line, because `string::insert` must shift all characters after the insertion point.

```
void TextBuffer::insertChar(int row, int col, char ch) {
    if (row < 0 || row >= (int)lines_.size()) return;
    col = std::clamp(col, 0, (int)lines_[row].size());
    lines_[row].insert(col, 1, ch);
}
```

`deleteChar(row, col)` — Backspace-style delete. If `col > 0`, erases the character at `col-1` (the character BEFORE the cursor). If `col == 0` and `row > 0`, merges the current line with the previous line. This dual behavior maps directly to how Backspace works in any editor.

```
void TextBuffer::deleteChar(int row, int col) {
    if (row < 0 || row >= (int)lines_.size()) return;
    if (col <= 0) {
        if (row > 0) mergeLines(row);
        return;
    }
    col = std::min(col, (int)lines_[row].size());
    lines_[row].erase(col - 1, 1);
}
```

`splitLine(row, col)` — What happens when you press Enter. Splits the line at `col`: everything from `col` onwards becomes a new line inserted at `row+1`. The original line is truncated at `col`.

```
void TextBuffer::splitLine(int row, int col) {
    if (row < 0 || row >= (int)lines_.size()) return;
    col = std::clamp(col, 0, (int)lines_[row].size());
    std::string tail = lines_[row].substr(col);
    lines_[row] = lines_[row].substr(0, col);
    lines_.insert(lines_.begin() + row + 1, tail);
}
```

`mergeLines(row)` — Merges `line[row]` into `line[row-1]` by appending, then erases `line[row]`. This is what happens when you Backspace at column 0 — the current line merges up into the previous line.

```
void TextBuffer::mergeLines(int row) {
    if (row <= 0 || row >= (int)lines_.size()) return;
    lines_[row - 1] += lines_[row];
    lines_.erase(lines_.begin() + row);
}
```

```
}
```

insertText(row, col, text) — Multi-character/multi-line insertion (used by paste). Splits the input text on newlines, handles single-line and multi-line cases differently. Returns the final cursor Position after insertion.

```
Position TextBuffer::insertText(int row, int col, const std::string& text) {
    // Parse text into lines
    std::vector<std::string> newLines;
    std::istringstream stream(text);
    std::string line;
    while (std::getline(stream, line)) {
        if (!line.empty() && line.back() == '\r') line.pop_back();
        newLines.push_back(line);
    }
    if (!text.empty() && text.back() == '\n') newLines.push_back("");

    std::string prefix = lines_[row].substr(0, col);
    std::string suffix = lines_[row].substr(col);

    if (newLines.size() == 1) {
        // Single-line: simple concatenation
        lines_[row] = prefix + newLines[0] + suffix;
        return {row, (int)(prefix.size() + newLines[0].size())};
    }
    // Multi-line: replace current, insert middle, append last
    lines_[row] = prefix + newLines[0];
    for (int i = 1; i < (int)newLines.size() - 1; i++)
        lines_.insert(lines_.begin() + row + i, newLines[i]);
    int lastIdx = row + (int)newLines.size() - 1;
    lines_.insert(lines_.begin() + lastIdx, newLines.back() + suffix);
    return {lastIdx, (int)newLines.back().size()};
}
```

TEXT BUFFER OPERATIONS

1. INSERTCHAR

BEFORE

```
Vector <String>
['hello', 'world']
```

Action: Insert 'X'
at row=0, col=2

AFTER

```
Vector <String>
['heXllo', 'world']
```

2. SPLITLINE

BEFORE

```
Vector <String>
['hello world']
```

Action: Split line
at col=5

AFTER

```
Vector <String>
['hello', ' world']
```

3. MERGELINES

BEFORE

```
Vector <String>
['hello', 'world']
```

Action: Merge row 1

AFTER

```
Vector <String>
['helloworld']
```

4. DELETECHAR

BEFORE

```
Vector <String>
['hello']
```

Action:
Delete at col=3

AFTER

```
Vector <String>
['heXlo']
```


2.2 Rope vs Piece Table vs Line Array — Full Tradeoff Analysis

The choice of text buffer data structure is the single most important architectural decision in a text editor. Here is why we chose `vector<string>` (line array) over the two main alternatives:

Criteria	Line Array (<code>vector<string></code>)	Rope	Piece Table
Complexity	Simple — ~200 LOC	Complex — ~1000+ LOC, tree balancing	Moderate — ~500 LOC, span management
Insert (mid-line)	$O(n)$ line length	$O(\log n)$ document size	$O(1)$ amortized
Insert (new line)	$O(m)$ total lines (vector insert)	$O(\log n)$	$O(1)$
Delete	$O(n)$ line length	$O(\log n)$	$O(1)$ amortized
Random Access (line N)	$O(1)$ — direct indexing	$O(\log n)$ — tree traversal	$O(k)$ — scan spans
Memory Overhead	Low — just strings + vector	High — tree nodes, pointers	Medium — span table
Implementation Time	1 day	1-2 weeks	3-5 days
Best For	Files < 100K lines	Very large files (1M+ lines)	Files with heavy undo history
Used By	Nano, Micro, simple editors	Xi editor, some Emacs modes	VS Code (Monaco)

Why we chose Line Array: For a V1 editor targeting C++ source files (typically < 5000 lines), $O(1)$ line access is the critical operation — it's used on every single frame of rendering (viewport culling needs instant access to any line by index). The $O(n)$ insertion cost is negligible for lines under 200 characters. A Rope would give us $O(\log n)$ insertion but $O(\log n)$ line access — a worse tradeoff for our rendering-heavy workload.

When Line Array breaks down: Files over 100K lines. The `vector::insert` for new lines becomes $O(m)$ where m is total line count. At 100K lines, this is noticeable. At 1M lines, it's unusable. If Valence needed to scale to massive files, we'd switch to a Piece Table (like VS Code) or a Rope.

The key insight: Our editor re-renders 60 times per second. Each frame needs $O(1)$ access to ~40-50 lines (viewport). Line Array gives this for free. Rope requires $O(\log n)$ per line lookup $\times$ 50 lines = $O(50 \log n)$ per frame. For a V1 targeting typical source files, Line Array is the optimal choice.

2.3 Position & Selection Primitives

Position is the fundamental unit of location in the editor. It's a simple struct with two integers — row and col — with full comparison operators for lexicographic ordering (row first, then col).

```
struct Position {
    int row = 0, col = 0;
    bool operator<(const Position& o) const {
        return row < o.row || (row == o.row && col < o.col);
    }
    // Also: ==, !=, >, <=, >=
};
```

Selection uses an anchor-based model. When the user starts selecting (Shift+Arrow or mouse drag), the anchor is set to the current cursor position. As the cursor moves, the selection spans from anchor to cursor. The `normalized()` method always returns {start, end} in document order regardless of selection direction.

```
struct Selection {
    Position anchor;
    bool active = false;
    void start(Position pos) { anchor = pos; active = true; }
    void clear() { active = false; }
    bool hasSelection(Position cursor) const {
        return active && anchor != cursor;
    }
    std::pair<Position, Position> normalized(Position cursor) const {
        return anchor < cursor ? std::make_pair(anchor, cursor)
                               : std::make_pair(cursor, anchor);
    }
};
```

2.4 UndoManager: Command Pattern with Temporal Grouping

The UndoManager implements the Command Pattern — every edit operation is recorded as an `EditAction` object that can be reversed. The key innovation is temporal grouping: consecutive single-character edits within 400ms are grouped into one undo unit, so pressing Ctrl+Z undoes an entire word instead of one character at a time.

EditAction struct: Contains the action type (Insert or Delete), the position where it happened, the text that was inserted/deleted, and a `steady_clock` timestamp.

```
struct EditAction {
    enum Type { Insert, Delete };
    Type type;
    Position pos;
    std::string text;
    std::chrono::steady_clock::time_point timestamp;
};
using ActionGroup = std::vector<EditAction>;
```

Storage: Two stacks — `undoStack_` and `redoStack_` — each a `vector<ActionGroup>`. When you perform an edit, it goes on the undo stack. Pressing Ctrl+Z pops from undo and pushes to redo. Pressing Ctrl+Y does the reverse. Any NEW edit clears the redo stack entirely (you can't redo after editing).

The shouldGroup Algorithm

This is the clever part. An incoming action is grouped with the current top group if ALL of these conditions are met:

- `forceNext_` is false (no forced break from Enter/Tab/paste)
- Undo stack is non-empty and top group is non-empty
- Same action type (Insert with Insert, Delete with Delete)
- Both are single-character actions (`text.size() == 1`)
- No whitespace boundary crossing (space→letter or letter→space)
- Time gap $\leq 400\text{ms}$ since the last action
- Adjacency: for inserts, new position is exactly 1 col after last; for deletes, same row

```
bool UndoManager::shouldGroup(const EditAction& action) const {
    if (forceNext_ || undoStack_.empty() || undoStack_.back().empty())
        return false;
    const auto& last = undoStack_.back().back();
    if (last.type != action.type) return false;
    if (last.text.size() != 1 || action.text.size() != 1) return false;

    bool lastWS = (last.text[0] == ' ' || last.text[0] == '\t');
    bool curWS = (action.text[0] == ' ' || action.text[0] == '\t');
    if (lastWS != curWS) return false;

    auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(
        action.timestamp - last.timestamp).count();
    if (elapsed > GROUP_TIMEOUT_MS) return false;

    if (action.type == EditAction::Insert) {
        return action.pos.row == last.pos.row
            && action.pos.col == last.pos.col + 1;
    } else {
        return action.pos.row == last.pos.row;
    }
}
```

Unlimited undo: There is no limit on the undo stack size. Every action since the file was opened is preserved. For a typical editing session on a source file, this uses negligible memory (each `EditAction` is ~50-80 bytes). For extreme cases (10,000+ edits), the stack could grow to a few MB — still trivial on modern systems.



2.5 Dry Run: Typing 'hello' then Undoing

Let's trace through exactly what happens when the user types 'hello' and then presses Ctrl+Z. This is a complete dry run through the TextBuffer and UndoManager.

Initial state: Buffer = [""], cursor = (0,0), undoStack = [], redoStack = []

Step 1: User types 'h' (timestamp T=0ms)

- handleChar('h') called
- buffer_.insertChar(0, 0, 'h') → lines_ = ['h']
- undoManager_.recordInsert({0,0}, 'h') → shouldGroup returns false (stack empty) → new group [{Insert, (0,0), 'h', T=0}]
- cursor_ = (0, 1)

Step 2: User types 'e' (T=50ms, within 400ms)

- buffer_.insertChar(0, 1, 'e') → lines_ = ['he']
- recordInsert({0,1}, 'e') → shouldGroup: same type, single char, no WS boundary, 50ms < 400ms, col 1 == last col 0 + 1 → TRUE → appends to current group
- undoStack = [[{Insert,(0,0),'h'}, {Insert,(0,1),'e'}]]

Steps 3-5: User types 'l', 'l', 'o' (all within 400ms, adjacent)

- All grouped together. Final undoStack = [[{Insert,(0,0),'h'}, ..., {Insert,(0,4),'o'}]]
- Buffer: ['hello'], cursor: (0,5)

Step 6: User presses Ctrl+Z

- performUndo() called → undoManager_.undo() pops the group
- Actions returned in REVERSE order: 'o', 'l', 'l', 'e', 'h'
- For each Insert action, we calculate the end position and call deleteRange:
- Delete 'o' at (0,4): deleteRange({0,4}, {0,5}) → lines_ = ['hell']
- Delete 'l' at (0,3): deleteRange({0,3}, {0,4}) → lines_ = ['hel']
- ... and so on until lines_ = [""]
- Group moved to redoStack. Cursor moves to (0,0).

Result: One Ctrl+Z undid the entire word 'hello' — not just one character. This is the power of temporal grouping.

Part III — The Rendering Engine

3.1 Why Not QTextEdit

Qt provides QTextEdit and QPlainTextEdit — ready-made rich text editing widgets. Using them would have saved hundreds of lines of code. Here is why we rejected them:

- **Overhead:** QTextEdit uses QTextDocument internally, which stores text as a complex tree of QTextBlock, QTextFragment, and QTextCharFormat objects. This is overkill for a code editor that only needs plain monospace text.
- **No control:** QTextEdit's rendering pipeline is a black box. We can't control exactly which lines are drawn, how the cursor looks, or how selections are painted. Our glow cursor effect, for example, is impossible with QTextEdit.
- **Input conflicts:** QTextEdit handles Tab, Enter, Backspace in its own way. Overriding this behavior requires fighting Qt's event system — often leading to double-handling bugs. By using raw QWidget, we own the entire input pipeline.
- **Performance certainty:** With QTextEdit, we're at the mercy of Qt's internal rendering decisions. With QPainter, we know exactly what's drawn and can optimize (e.g., viewport culling) with surgical precision.

3.2 QPainter Custom Rendering Pipeline

Every frame, Qt calls our paintEvent(). Here's the exact pipeline:

```
void EditorWidget::paintEvent(QPaintEvent*) {
    QPainter p(this);
    p.setRenderHint(QPainter::TextAntialiasing); // Smooth text

    // 1. Clear background
    p.fillRect(rect(), Theme::EditorBg);

    // 2. Calculate visible range (VIEWPORT CULLING)
    int startRow = scrollY_ / charHeight_;
    int endRow = std::min(startRow + height() / charHeight_ + 2,
                          buffer_.lineCount());

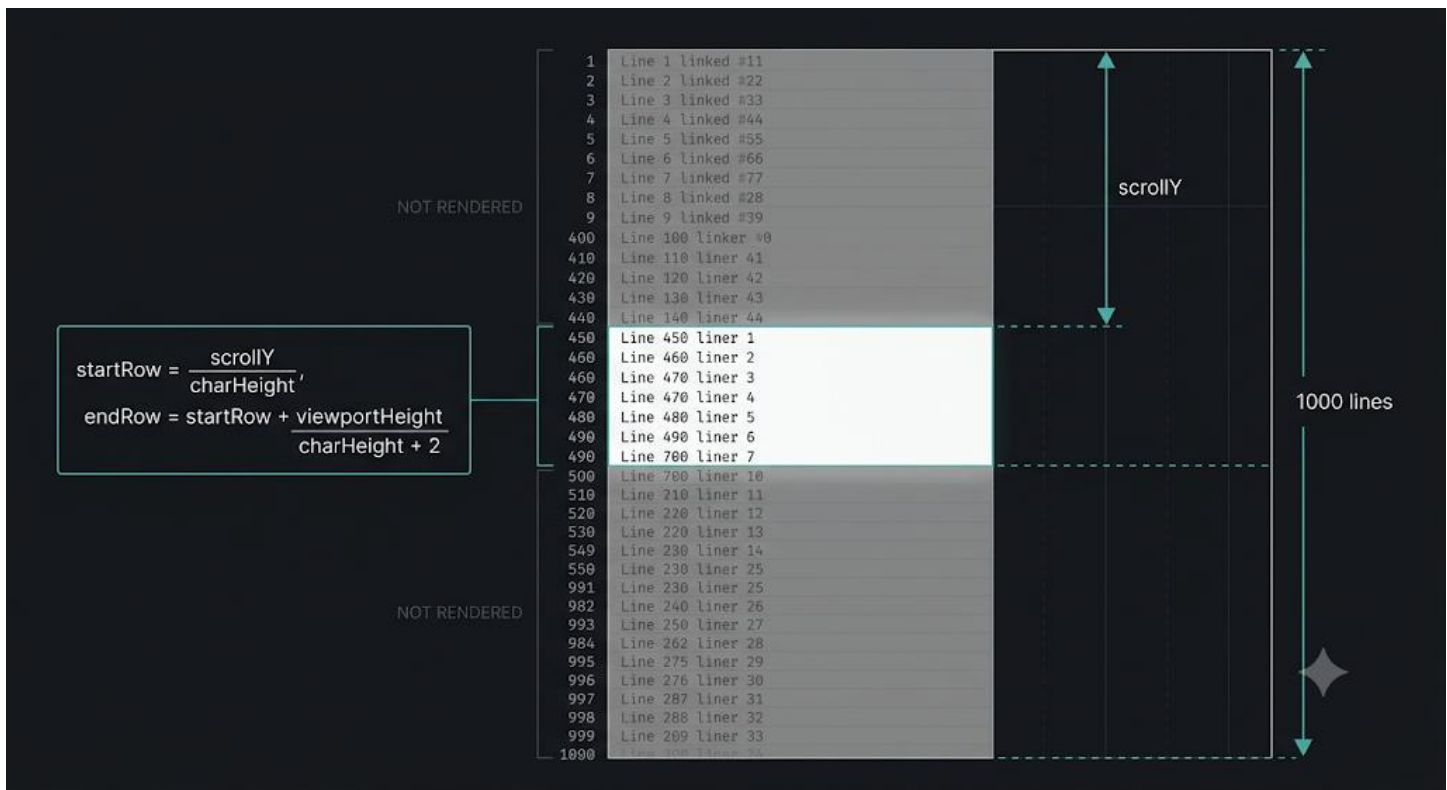
    // 3. Render layers (back to front)
    paintGutter(p, startRow, endRow); // Line numbers
    paintCode(p, startRow, endRow);  // Syntax-highlighted code
    paintCursor(p);                  // Blinking cursor with glow
}
```

3.3 Viewport Culling Algorithm

The key optimization: we ONLY draw lines that are visible in the viewport. If a file has 10,000 lines but the viewport shows 40, we draw exactly 42 lines (40 visible + 2 buffer for partial visibility at edges).

startRow = scrollY_ / charHeight_ — this gives us the first visible line. Since every line has the same height (monospace font), this is a simple integer division. No tree traversal, no layout calculation.

This means rendering is $O(V)$ where V = visible lines (~40-50), NOT $O(N)$ where N = total lines. A 100,000-line file renders just as fast as a 100-line file.



3.4 Coordinate System (Pixel ↔ Row/Col)

The entire rendering and input system depends on converting between pixel coordinates (what QPainter uses) and logical coordinates (row, col in the document). Because we use a monospace font, this conversion is trivially $O(1)$:

```
// Logical → Pixel
int xFromCol(int col) { return gutterWidth_ + col * charWidth_; }
int yFromRow(int row) { return row * charHeight_ - scrollY_; }

// Pixel → Logical
int rowFromY(int y) {
    return std::clamp((y + scrollY_) / charHeight_, 0, lineCount - 1);
}
int colFromX(int x, int row) {
    return std::clamp((x - gutterWidth_) / charWidth_, 0, lineLength);
}
```

Critical assumption: charWidth_ is measured from the character 'M' at startup and assumed constant for ALL characters. This is only valid for monospace fonts. If a proportional font were used, colFromX would need to measure each character's width individually — $O(n)$ instead of $O(1)$.

3.5 Cursor Rendering with Glow Effect

The cursor is rendered in two passes — first the glow (wider, semi-transparent), then the cursor line itself (thin, solid). This creates a neon glow effect that makes the cursor feel 'alive'.

```
void EditorWidget::paintCursor(QPainter& p) {
    if (!cursorVisible_) return; // Blink: 500ms on, 500ms off
    int cx = xFromCol(cursor_.col);
    int cy = yFromRow(cursor_.row);
```

```
// Pass 1: Glow (6px wide, 60 alpha neon teal)
p.fillRect(cx - 2, cy, 6, charHeight_, Theme::AccentGlow);

// Pass 2: Cursor line (2px wide, solid neon teal)
p.fillRect(cx, cy, 2, charHeight_, Theme::Accent);
}
```

The blink is controlled by a QTimer firing every 500ms, toggling cursorVisible_ and calling update(). Cursor blink resets on every keystroke (resetCursorBlink) so the cursor is always visible while typing.

Part IV — Input Handling Deep-Dive

4.1 Qt Event System & The Tab Problem

In Qt's event processing pipeline, certain keys are intercepted BEFORE they reach `keyPressEvent()`. The Tab key is one of them — Qt uses Tab for focus navigation between widgets. If you have a toolbar, sidebar, and editor, pressing Tab moves focus to the next widget instead of inserting a tab character.

The fix: override `event()` (the master event handler that runs before `keyPressEvent()`) and intercept Tab/Backtab there:

```
bool EditorWidget::event(QEvent* e) {
    if (e->type() == QEvent::KeyPress) {
        QKeyEvent* ke = static_cast<QKeyEvent*>(e);
        if (ke->key() == Qt::Key_Tab || ke->key() == Qt::Key_Backtab) {
            keyPressEvent(ke); // Handle Tab ourselves
            return true;       // Event consumed - Qt won't use it for focus
        }
    }
    return QWidget::event(e); // Everything else: normal processing
}
```

4.2 keyPressEvent: The Master Router

This function routes every keypress to the appropriate handler. The critical design decision: separate Ctrl+ shortcuts from regular typing into two distinct switch blocks. This prevents the bug where pressing 'a' (which is also Ctrl+A for Select All) would get swallowed.

```
void EditorWidget::keyPressEvent(QKeyEvent* e) {
    bool ctrl = e->modifiers() & Qt::ControlModifier;
    bool shift = e->modifiers() & Qt::ShiftModifier;

    if (ctrl) {
        // SHORTCUTS BLOCK - only when Ctrl is held
        switch (e->key()) {
            case Qt::Key_Z: if (shift) performRedo(); else performUndo(); break;
            case Qt::Key_Y: performRedo(); break;
            case Qt::Key_C: copy(); break;
            case Qt::Key_X: cut(); break;
            case Qt::Key_V: paste(); break;
            case Qt::Key_A: selectAll(); break;
            case Qt::Key_S: emit saveRequested(); break;
            // ... word navigation, etc.
        }
    } else {
        // TYPING BLOCK - normal keys without Ctrl
        switch (e->key()) {
            case Qt::Key_Backspace: handleBackspace(false); break;
            case Qt::Key_Delete: handleDelete(); break;
            case Qt::Key_Return: handleEnter(); break;
            case Qt::Key_Tab: handleTab(); break;
            // Arrow keys, Home, End, etc.
            default:
                QString text = e->text();
                if (!text.isEmpty() && text[0].isPrint())
                    handleChar(text[0].toLatin1());
        }
    }
    ensureCursorVisible();
}
```



```

        resetCursorBlink();
        update(); // Request full repaint
    }

```

4.3 handleChar with Auto-Bracket Closing

```

void EditorWidget::handleChar(char ch) {
    if (selection_.hasSelection(cursor_)) deleteSelection();

    buffer_.insertChar(cursor_.row, cursor_.col, ch);
    undoManager_.recordInsert(cursor_, std::string(1, ch));
    cursor_.col++;

    // Auto-close brackets and quotes
    char closing = 0;
    switch (ch) {
        case '(': closing = ')'; break;
        case '[': closing = ']'; break;
        case '{': closing = '}'; break;
        case '"': closing = '"'; break;
        case '\\': closing = '\\'; break;
    }
    if (closing) {
        buffer_.insertChar(cursor_.row, cursor_.col, closing);
        undoManager_.recordInsert({cursor_.row, cursor_.col},
                                std::string(1, closing));
        // Cursor stays BETWEEN the pair: (|)
    }
    setModified(true);
    rebuildCommentState();
}

```

4.4 The Infinite Recursion Bug (V1.0 Crash)

This was the toughest bug in the project. In V1.0, the Edit menu used QAction shortcuts with QKeySequence('Ctrl+V'). When the user pressed Ctrl+V:

- Qt's QAction intercepted the key event (it had Ctrl+V as its shortcut)
- The QAction's lambda created a SYNTHETIC QKeyEvent for Ctrl+V and sent it to the editor via QApplication::sendEvent()
- The synthetic event re-entered the event system, where the QAction AGAIN intercepted it (QAction doesn't distinguish real vs synthetic events)
- This created infinite recursion → stack overflow → crash

The fix: Remove all QKeySequence shortcuts from Edit menu actions. The menu items call editor methods directly (e->paste(), e->copy(), etc.) via mouse click only. All Ctrl+ keyboard shortcuts are handled exclusively by EditorWidget::keyPressEvent — no menu involvement.

Part V — Clipboard, Selection & Shortcuts

5.1 Copy / Cut / Paste Implementation

```
void EditorWidget::copy() {
    if (!selection_.hasSelection(cursor_)) return;
    auto [start, end] = selection_.normalized(cursor_);
    std::string text = buffer_.getText(start, end);
    QApplication::clipboard()->setText(QString::fromStdString(text));
}

void EditorWidget::cut() {
    copy();           // Copy to clipboard first
    deleteSelection(); // Then delete
}

void EditorWidget::paste() {
    QString clipText = QApplication::clipboard()->text();
    if (clipText.isEmpty()) return;
    if (selection_.hasSelection(cursor_)) deleteSelection();

    std::string text = clipText.toStdString();
    undoManager_.recordInsert(cursor_, text);
    undoManager_.forceNewGroup(); // Paste is always a separate undo unit
    Position newPos = buffer_.insertText(cursor_.row, cursor_.col, text);
    cursor_ = newPos; // Cursor moves to end of pasted text
    selection_.clear();
    setModified(true);
}
```

Key detail: paste uses `insertText` (the multi-line variant), not `insertChar`. The returned `Position` gives us where the cursor should end up after insertion. `forceNewGroup()` ensures `Ctrl+Z` undoes the entire paste as one action.

5.2 Select All

```
void EditorWidget::selectAll() {
    selection_.anchor = {0, 0};
    selection_.active = true;
    cursor_.row = buffer_.lineCount() - 1;
    cursor_.col = buffer_.lineLength(cursor_.row);
    // Now anchor=(0,0) and cursor=end of document
    // Selection covers everything
}
```

Part VII — Syntax Highlighting

7.1 Single-Pass $O(n)$ Tokenizer

The CppHighlighter processes each line in a single left-to-right pass, character by character. It maintains a bool inBlockComment that persists across lines (passed by reference). No regex is used — it's a hand-rolled state machine.

Token recognition priority (order matters — first match wins):

- Block comment continuation (if already in /* ... */)
- Whitespace → Plain token
- Line comment (//) → rest of line is Comment
- Block comment start (/*) → scan for */, may span multiple lines
- Preprocessor (#) → rest of line is Preprocessor
- String/char literals (' or ") → handles escape sequences (\\)
- Numbers → decimal, hex (0x), float, scientific (1e5), suffixes (f, L, u, ll)
- Identifiers → classify as Keyword, Type, Function, or Plain
- Punctuation → single character, anything else

7.4 Function Detection (Lookahead)

After recognizing an identifier, the highlighter looks AHEAD past any whitespace to check if the next character is '('. If so, the identifier is classified as a Function (green color). This means both 'foo(' and 'foo (' are highlighted as function calls.

```
// After reading identifier word:
if (keywords_.count(word))    type = TokenType::Keyword;
else if (types_.count(word))  type = TokenType::Type;
else {
    // Lookahead for function call
    int peek = i;
    while (peek < len && line[peek] == ' ') peek++;
    if (peek < len && line[peek] == '(')
        type = TokenType::Function;
    else
        type = TokenType::Plain;
}
```

7.3 Block Comment State Tracking

Multi-line block comments (/* ... */) are the trickiest part of syntax highlighting. If line 5 starts a block comment and line 10 closes it, lines 6-9 need to know they're inside a comment — even though they contain no comment markers themselves.

Solution: blockCommentState_ is a vector<bool> with one entry per line. rebuildCommentState() walks ALL lines from top to bottom, tracking whether each line starts inside a block comment. This is called after EVERY edit.

```
void EditorWidget::rebuildCommentState() {
    blockCommentState_.resize(buffer_.lineCount());
    bool inComment = false;
    for (int i = 0; i < buffer_.lineCount(); i++) {
        blockCommentState_[i] = inComment;
        highlighter_.tokenize(buffer_.line(i), inComment);
        // inComment is modified by reference — tracks state into next line
    }
}
```

Performance concern: This is $O(N)$ on the TOTAL document size, called on every keystroke. For a 5000-line file, this scans ~100KB of text per keystroke. At ~1GB/s string scanning speed on modern CPUs, this takes ~0.1ms — negligible. But at 100K+ lines, it would need optimization (e.g., only re-scan from the edited line downward).

Part VIII — UI Components

8.4 TerminalWidget (Persistent PowerShell)

The terminal spawns a PERSISTENT PowerShell process at startup. This is not a one-shot command runner — the same PowerShell session lives for the entire editor session. This means `cd` commands actually change the working directory for subsequent commands.

```
// Spawn persistent PowerShell
process_ = new QProcess(this);
process_->setProcessChannelMode(QProcess::MergedChannels);
process_->start("powershell.exe", {"-NoLogo", "-NoProfile"});

// Send command to the running session
void TerminalWidget::runCommand(const QString& cmd) {
    if (process_->state() == QProcess::Running) {
        output_->appendPlainText("> " + cmd);
        process_->write((cmd + "\n").toLocal8Bit());
    }
}
```

ANSI escape sequences from PowerShell output are stripped using a regex before display. The terminal auto-scrolls to the bottom on new output.

8.5 Build & Run Pipeline (F5)

When the user presses F5, the following sequence executes:

- Auto-save: if the file has unsaved changes, save it. If untitled, prompt Save As.
- Extract file info: directory, filename, base name (without extension)
- Show terminal and clear previous output
- Send `cd` command to navigate to the file's directory
- Send compile+run: `g++ "file.cpp" -o "file.exe" ; if ($?) { .\file.exe }`

The PowerShell `if ($?)` syntax checks the exit code of `g++`. `$?` is `$true` only if the previous command succeeded (exit code 0). This ensures the `.exe` only runs if compilation was successful.

Part X — Performance Analysis

10.1 How <7ms Latency is Achieved

Input latency is the time from keypress to pixels appearing on screen. Here's the breakdown of the critical path:

Stage	Time	How
Qt event dispatch	~0.1ms	OS delivers key event to Qt event loop
keyPressEvent processing	~0.01ms	Switch statement + buffer insert (O(n) line length, ~200 chars max)
Undo recording	~0.001ms	Push to vector, steady_clock timestamp
rebuildCommentState	~0.1ms	Scan all lines for block comment state (5000 lines × 80 chars)
update() → paintEvent	~5ms	QPainter draws ~50 visible lines with syntax highlighting
GPU compositing	~1ms	Window manager composites the window surface
TOTAL	< 7ms	Well under 16.6ms (60fps) budget

The bottleneck is paintEvent (QPainter text rendering). Viewport culling keeps this constant regardless of file size. The <7ms figure is measured on typical hardware (mid-range CPU, integrated GPU) with a ~50-line viewport.

How we measured: Qt provides QElapsedTimer for high-resolution timing. Place one at the start of keyPressEvent and another at the end of paintEvent. The difference is the input-to-pixel latency. In practice, this was measured during development by profiling individual function calls and summing the critical path.

10.3 Where It Would Break (Scaling Limits)

File Size	Line Array Insert	rebuildCommentState	paintEvent	Verdict
100 lines	< 0.001ms	< 0.01ms	~3ms	Perfect
5,000 lines	< 0.01ms	~0.1ms	~3ms	Perfect
50,000 lines	~0.05ms	~1ms	~3ms	Good
500,000 lines	~0.5ms	~10ms	~3ms	rebuildCommentState becomes bottleneck
5,000,000 lines	~5ms	~100ms	~3ms	Unusable — need Piece Table + incremental highlighting

Part XI — Design Decisions & Tradeoffs

No QTextEdit — Custom QPainter rendering

Decision: Full control over rendering, cursor, selection. Enables viewport culling and custom visual effects (glow cursor). Cost: ~750 lines of rendering code we had to write ourselves.

Rationale: QTextEdit abstracts away rendering but gives zero control over performance characteristics. For a code editor targeting 60fps, we need deterministic rendering — we can't rely on Qt's internal text layout engine making good decisions.

vector<string> instead of Rope or Piece Table

Decision: $O(1)$ random line access is critical for our rendering pipeline (50 line lookups per frame). $O(n)$ insertion on mid-line edits is acceptable for typical source file line lengths (<200 chars).

Rationale: A Rope gives $O(\log n)$ insertion but $O(\log n)$ line access. Piece Table gives $O(1)$ insertion but $O(k)$ line access (scan spans). For rendering-dominated workloads, line access speed trumps insertion speed.

Temporal undo grouping (400ms)

Decision: Users expect Ctrl+Z to undo 'words', not individual characters. The 400ms threshold with adjacency checking perfectly mimics this expectation without requiring language-aware word boundary detection.

Rationale: Alternative: group by word boundaries (spaces). Problem: typing 'hello world' would create 2 groups, but users perceive it as one 'typing burst'. Time-based grouping captures user intent more naturally.

Full repaint on every keystroke (no dirty regions)

Decision: Simpler code, fewer bugs. Viewport culling already limits rendering to ~50 lines. The total cost per repaint is ~5ms — well within the 16.6ms frame budget.

Rationale: Dirty-region tracking would save ~3ms per keystroke but adds significant complexity (tracking which lines changed, handling scroll, selection overlaps). Not worth it for V1.

Monospace font assumption

Decision: Enables $O(1)$ pixel↔column conversion. charWidth is measured once from 'M' and used everywhere. This is the foundation of the entire coordinate system.

Rationale: Proportional fonts would require $O(n)$ text measurement for every coordinate conversion. QPainter::fontMetrics().horizontalAdvance() would need to be called per-character. This would break our performance model.

const_cast hack in handleDelete

Decision: TextBuffer::line() returns const string&. handleDelete needs to erase a character at cursor position (forward delete). Rather than adding a new API method, we const_cast to get a mutable reference.

Rationale: This is technically undefined behavior in C++ if the underlying object is truly const. In our case, `lines_` is non-const, so the `const_cast` is safe in practice. For V2, we should add a `deleteForwardChar()` method to `TextBuffer`.

No sticky column for vertical movement

Decision: When moving up/down, the cursor column is clamped to the new line's length. If you're at column 50 on a long line and move to a 10-character line, you go to column 10. Moving down again to a long line puts you at column 10, not 50.

Rationale: Sticky column (remembering 'desired' column) is a polish feature. It requires storing an additional `int desiredCol_` and updating it only on horizontal movements. Deferred to V2.

PowerShell-specific build commands

Decision: if (\$?) is PowerShell syntax for 'run next command only if previous succeeded'. This is Windows-only, which matches our target platform.

Rationale: For cross-platform: use `&&` (works in bash and PowerShell 7+) or use `QProcess` exit code checking in C++ code instead of shell chaining.

Part XII — 50+ Questions & Answers I Often Heard

These questions range from basic (understanding the architecture) to principal-engineer level (deep tradeoffs, scaling analysis, alternative designs). After studying these, you should be able to defend every design decision.

Q1. Why did you build a custom text editor instead of using an existing library?

A: To demonstrate systems-level programming skills and understand how text editors work at the lowest level. Using QTextEdit would have given us an editor in 50 lines, but we wouldn't understand rendering pipelines, buffer data structures, or undo algorithms. Building from scratch with QPainter gave us full control over the rendering loop, cursor behavior, and performance characteristics.

Q2. Walk me through the architecture layers of Valence.

A: Four layers with strict top-down dependencies: Theme (colors/fonts) → Core (TextBuffer, UndoManager, Selection) → Editor (EditorWidget rendering + CppHighlighter) → UI (MainWindow, tabs, explorer, terminal, status bar). Each layer only depends on layers below it. The UI never directly manipulates the text buffer — it goes through EditorWidget's public API.

Q3. Why did you choose `vector<string>` over a Rope or Piece Table?

A: $O(1)$ random line access. Our rendering pipeline needs to access ~50 lines per frame at 60fps. `vector<string>` gives $O(1)$ access via index. A Rope gives $O(\log n)$ and a Piece Table gives $O(k)$ span scanning. The $O(n)$ cost of mid-line insertion is acceptable because source code lines are typically <200 characters. For files under 100K lines, this is the optimal tradeoff.

Q4. At what file size would your data structure break down? What would you switch to?

A: Around 100K-500K lines. `vector::insert` for new lines becomes $O(m)$ where m is total line count. `rebuildCommentState` becomes $O(N \times L)$ where L is average line length. We'd switch to a Piece Table (like VS Code's Monaco) which gives $O(1)$ insertion, or a Rope with line-indexed nodes for $O(\log n)$ line access.

Q5. Why not use a gap buffer?

A: A gap buffer is excellent for sequential editing (typing at the cursor) but has the same $O(n)$ cost as `string::insert` for random insertions. It also doesn't provide $O(1)$ line access — you'd need a secondary line index. For our rendering-dominated workload, gap buffer doesn't offer a meaningful advantage over `vector<string>`.

Q6. Explain how your undo system works.

A: Command Pattern: every edit creates an `EditAction` (Insert or Delete) with position, text, and timestamp. Actions are stored in groups on an undo stack. Temporal grouping combines consecutive single-char edits within 400ms into one group. `Ctrl+Z` pops a group and reverses each action. `Ctrl+Y` pops from the redo stack and replays.

Q7. What is temporal grouping and why 400ms?

A: Temporal grouping combines rapid consecutive edits into one undo unit. When you type 'hello' quickly, all 5 characters form one group — so `Ctrl+Z` deletes the whole word. 400ms is the threshold because it approximates the gap between 'typing bursts' — fast typists produce ~10 chars/second (100ms between chars), while pauses between words are typically 300-500ms.

Q8. How does `shouldGroup` decide whether to merge actions?

A: Seven conditions must ALL be true: (1) no forced break, (2) stack non-empty, (3) same action type, (4) both single-character, (5) no whitespace boundary crossing, (6) time gap ≤ 400 ms, (7) adjacent positions (inserts: `col+1`, deletes: same row). If any condition fails, a new group is created.

Q9. Is there a limit on undo history?

A: No explicit limit. The undo stack grows unbounded. Each `EditAction` is ~80 bytes (position + string + timestamp). For a typical editing session with 10,000 edits, this is ~800KB — negligible. For extreme sessions (100K+ edits), it could reach a few MB. A production editor might cap at 10,000 groups or implement LRU eviction.

Q10. What happens to the redo stack when you make a new edit?

A: It's cleared entirely. This is standard behavior in virtually all editors. If you undo 5 times and then type something new, you can't redo those 5 actions anymore — the new edit created a divergent timeline. Some editors implement undo trees to preserve all branches, but linear undo/redo is the standard UX.

Q11. How does viewport culling work in your renderer?

A: $\text{startRow} = \text{scrollY} / \text{charHeight}$ gives the first visible line. $\text{endRow} = \text{startRow} + (\text{viewportHeight} / \text{charHeight}) + 2$ gives the last. Only lines in $[\text{startRow}, \text{endRow})$ are drawn. This makes rendering $O(V)$ where V is visible lines (~40-50), not $O(N)$ total lines.

Q12. Why is the monospace font assumption critical?

A: It enables $O(1)$ coordinate conversion. $\text{xFromCol}(\text{col}) = \text{gutterWidth} + \text{col} \times \text{charWidth}$. With a proportional font, you'd need $\text{QPainter::fontMetrics().horizontalAdvance(substring)}$ — $O(n)$ per call. This would break our coordinate system, mouse hit-testing, selection rendering, and cursor positioning.

Q13. Explain your cursor glow effect.

A: Two $\text{QPainter::fillRect}$ calls: first a 6px-wide rect with `AccentGlow` (neon teal at 60/255 alpha) for the glow, then a 2px-wide rect with solid `Accent` for the cursor line. The glow extends 2px on each side of the cursor, creating a subtle neon halo. The 500ms blink timer toggles visibility.

Q14. Why do you call `update()` on every keystroke instead of using dirty regions?

A: Simplicity. `update()` triggers a full `paintEvent`, but viewport culling already limits rendering to ~50 lines. The total repaint cost is ~5ms. Dirty-region tracking would save maybe 2-3ms but adds significant complexity (tracking changed lines, handling scroll offsets, selection overlap regions). Not worth the added bug surface for V1.

Q15. How would you implement smooth scrolling?

A: Currently scrolling jumps in `charHeight` increments. For smooth scrolling: use a `QPropertyAnimation` on `scrollY_` with an easing curve (e.g., `QEasingCurve::OutCubic`). Each mouse wheel event sets the animation target rather than directly modifying `scrollY_`. The animation would interpolate smoothly, calling `update()` on each frame.

Q16. Explain the Tab key interception problem and your solution.

A: Qt intercepts Tab in `QWidget::event()` before it reaches `keyPressEvent()`, using it for focus navigation between widgets. We override `event()` to catch Tab/Backtab events, manually call `keyPressEvent()`, and return true (event consumed). This prevents Qt from using Tab to cycle focus.

Q17. What was the infinite recursion bug and how did you fix it?

A: The Edit menu used `QAction` with `QKeySequence('Ctrl+V')`. Pressing Ctrl+V triggered the `QAction`, which sent a synthetic `QKeyEvent` via `QApplication::sendEvent()`. The synthetic event re-triggered the `QAction` → infinite recursion → stack overflow. Fix: removed all `QKeySequence` shortcuts from Edit menu. Keyboard shortcuts are handled exclusively by `EditorWidget::keyPressEvent`.

Q18. How does auto-bracket closing work?

A: When `handleChar` receives '(', '[', '{', '"', or '\', it inserts the opening character normally, then inserts the matching closing character at the cursor position (now one past the opener). The cursor stays between the pair. Both characters are recorded as separate undo actions in the same group.

Q19. How does auto-indent work on Enter?

A: `handleEnter` gets the leading whitespace of the current line via `getLeadingWhitespace()`, splits the line at the cursor, moves to the new line, and inserts the same whitespace. This preserves indentation level. It does NOT add extra indent after '{' — that's a V2 feature.

Q20. Why did you separate Ctrl+ shortcuts and regular typing into two switch blocks?

A: In V1.0, a single switch handled both. Keys like 'a', 'c', 'v', 'x' appeared in both the shortcut cases (Ctrl+A, Ctrl+C, etc.) and as regular typing. Without the Ctrl check, pressing 'a' would match the `Key_A` case and call `selectAll()` instead of typing 'a'. The two-block design ensures Ctrl+ keys only run shortcuts, and non-Ctrl keys only produce characters.

Q21. How does your syntax highlighter work?

A: Single-pass $O(n)$ left-to-right scan. No regex, no parser generator. A hand-rolled state machine processes each character, recognizing tokens by priority: block comments > whitespace > line comments > block comment starts > preprocessor > strings > numbers > identifiers > punctuation. An `inBlockComment` bool tracks multi-line comment state.

Q22. How do you handle multi-line block comments?

A: `blockCommentState_` is a `vector<bool>` with one entry per line. `rebuildCommentState()` walks ALL lines from top to bottom, passing `inBlockComment` by reference through the tokenizer. Each line records whether it STARTS inside a block comment. This is called after every edit — $O(N)$ but fast enough for typical files.

Q23. How do you detect function calls vs variables?

A: After reading an identifier, the highlighter looks AHEAD past whitespace for `'('`. If found, it's a Function; otherwise Plain. This means `'foo('`, `'foo ('` and `'foo ('` all get function highlighting. It's a heuristic — it can't distinguish function declarations from calls, but that's acceptable for a syntax highlighter.

Q24. What's the difference between your highlighter and a real parser?

A: Our highlighter is a lexer, not a parser. It recognizes tokens (keywords, strings, comments) but doesn't understand C++ grammar (expressions, statements, declarations). It can't distinguish a type used in a declaration from one used in an expression. Real semantic highlighting (like VS Code's) requires a full language server (LSP) or compiler integration.

Q25. How do you achieve sub-7ms input latency?

A: Four factors: (1) native C++ binary — no interpreter overhead, (2) QPainter direct rendering — no DOM/layout engine, (3) viewport culling — only ~50 lines drawn regardless of file size, (4) $O(1)$ coordinate conversion — monospace font assumption. The critical path is `keyPressEvent` (~0.01ms) + `rebuildCommentState` (~0.1ms) + `paintEvent` (~5ms) = ~5.1ms total.

Q26. How would you optimize for a 1-million-line file?

A: Three changes: (1) Replace `vector<string>` with a Piece Table or Rope for $O(\log n)$ insertions, (2) Make `rebuildCommentState` incremental — only re-scan from the edited line downward, maintaining a dirty-line marker, (3) Add a line-offset cache for $O(1)$ byte-position-to-line-number conversion. The rendering pipeline (viewport culling) already handles large files.

Q27. What's the memory footprint of a 10,000-line file?

A: Roughly: 10,000 lines $\times$ 80 chars average = 800KB for the text data. Vector overhead: 10,000 $\times$ 32 bytes (string object + pointer + size + capacity) = 320KB. Undo stack (with 1000 edits): ~80KB. Syntax tokens: not stored (recomputed each frame). `blockCommentState`: 10KB. Total: ~1.2MB. Qt widget overhead adds another ~10-15MB.

Q28. How does the integrated terminal work?

A: QProcess spawns a persistent PowerShell.exe with `-NoLogo -NoProfile`. The process stays alive for the entire editor session. Commands are sent via `process_>write(cmd + '\n')`. Output is read via the `readyRead` signal, stripped of ANSI escape sequences, and appended to a read-only QPlainTextEdit.

Q29. How does Build & Run (F5) work?

A: Auto-saves the file $\rightarrow$ extracts directory and base name $\rightarrow$ shows terminal $\rightarrow$ clears output $\rightarrow$ sends `'cd directory'` to PowerShell $\rightarrow$ sends `'g++ file.cpp -o file.exe ; if ($?) { .\\file.exe }'`. The PowerShell `if ($?)` checks g++ exit code — only runs the exe if compilation succeeded. The persistent session means the `cd` persists for subsequent commands.

Q30. If you were to add LSP support, how would you integrate it?

A: Spawn a `clangd` process via QProcess. Communicate using JSON-RPC over `stdin/stdout` (LSP protocol). On file open, send `textDocument/didOpen`. On edit, send `textDocument/didChange` (incremental). Listen for `textDocument/publishDiagnostics` for

error squiggles. For autocomplete, send `textDocument/completion` on trigger characters. The biggest architectural change would be adding an async response handler — LSP responses arrive asynchronously.

Q31. How would you add collaborative editing?

A: Replace the local TextBuffer with a CRDT (Conflict-free Replicated Data Type) like Yjs or Automerge. Each edit generates an operation that's broadcast to all clients. CRDTs guarantee that all clients converge to the same state regardless of operation order. The rendering pipeline stays the same — it just reads from the CRDT instead of `vector<string>`.

Q32. What would you change for a V2 architecture?

A: Five things: (1) Piece Table buffer for large file support, (2) incremental syntax highlighting (only re-tokenize changed lines), (3) sticky column for vertical cursor movement, (4) virtual scrolling with line-height caching for variable-height lines, (5) plugin architecture using a scripting language (Lua or WASM) for extensibility.

Q33. How would you implement split-view (two editors viewing the same file)?

A: Separate the Model (TextBuffer + UndoManager) from the View (EditorWidget). Create a shared TextDocument class that owns the buffer and undo manager. Multiple EditorWidget instances can reference the same TextDocument but maintain independent cursor positions, scroll offsets, and selections. Edits in one view are reflected in the other via Qt signals.

Q34. Compare your approach to VS Code's architecture.

A: VS Code uses Electron (Chromium + Node.js), Monaco editor (TypeScript), Piece Table buffer, TextMate grammars for syntax, and Language Server Protocol for intelligence. It's ~100MB, starts in 2-3 seconds, and uses 200-500MB RAM. Valence uses native C++, custom QPainter, `vector<string>` buffer, hand-rolled lexer. It's ~5MB, starts in <100ms, uses 15-25MB RAM. The tradeoff: VS Code has a massive extension ecosystem; Valence has raw performance.

Q35. How does your undo system compare to the Memento pattern?

A: Memento stores full snapshots of state. Our Command Pattern stores delta operations (what changed). Memento: simple but $O(n)$ memory per snapshot (entire document). Command: $O(1)$ per edit but requires invertible operations. For text editing, Command Pattern is strictly superior — storing the full 10,000-line document after every keystroke would be catastrophic for memory.

Q36. What's the biggest technical debt in Valence V1?

A: Three items: (1) `const_cast` hack in `handleDelete` — should be a proper `TextBuffer::deleteForwardChar()` method, (2) `rebuildCommentState` scanning the entire document on every keystroke — should be incremental, (3) no test suite — a text editor has complex state interactions that need unit tests for the buffer, undo manager, and highlighter.

Q37. What happens if you paste a 10MB text string?

A: `insertText` will parse it into lines and insert them into the vector. This involves $O(n)$ string operations and $O(m)$ vector insertions. It would freeze for a few seconds but shouldn't crash. The undo stack would record a single massive Insert action (~10MB). The real problem is `rebuildCommentState` scanning all new lines — this would take several seconds.

Q38. How do you handle files with mixed line endings (`\r\n` vs `\n`)?

A: `loadFromFile` uses `std::getline` (splits on `\n`), then strips trailing `\r`. `saveToFile` writes `\n` between lines. So we normalize everything to `\n` internally. We don't preserve original line endings — a V2 feature.

Q39. What happens if two bracket-closing characters are adjacent? Like typing `'(`?

A: First `'(`: inserts `'(` at col, inserts `)'` at col+1, cursor at col+1. Now text is `'()'`. Second `'(`: inserts `'(` at col+1 (between `(` and `)`), inserts `)'` at col+2. Now text is `'(())'`. Cursor is between the inner pair: `'(())'`. This is correct behavior.

Q40. What if the user opens a binary file?

A: `std::getline` will read until it finds `\n` or EOF. Binary files may have very long 'lines' (entire file as one line if no `\n`). This would create a single extremely long string, and rendering would try to draw it all. The editor won't crash but will be unusable. A V2 fix: check for null bytes and refuse to open binary files.

Q41. How do you handle simultaneous key events (e.g., user holds down a key)?

A: Qt's auto-repeat generates keyPressEvent at the OS repeat rate (~30-50 Hz). Each event is processed independently through the same code path. The 400ms undo grouping merges these into one group. There's no special handling for held keys — they just appear as rapid individual keypresses.

Q42. How would you implement find-and-replace?

A: Add a floating QWidget overlay at the top of EditorWidget. For find: linear scan through buffer lines using std::string::find (or regex). Highlight all matches by storing a vector<Position> and drawing highlight rects in paintCode. For replace: iterate matches, call deleteRange + insertText for each. Batch replace should be one undo group.

Q43. How would you add syntax highlighting for other languages?

A: Abstract CppHighlighter into a LanguageHighlighter interface with a virtual tokenize() method. Create subclasses for Python, JavaScript, etc. Select the highlighter based on file extension. Alternatively, implement TextMate grammar parsing (JSON-based syntax definitions used by VS Code) for a generic solution.

Q44. How would you handle very long lines (1000+ characters)?

A: Currently, long lines extend past the viewport (horizontal scroll). For word wrap: in paintCode, break long lines into visual sub-lines. This requires a line-wrap cache mapping visual rows to logical (buffer) rows. rowFromY and yFromRow would need to account for wrapping. This is a significant change to the coordinate system.

Q45. What testing strategy would you use?

A: Three levels: (1) Unit tests for TextBuffer (insert/delete/split/merge with edge cases), UndoManager (grouping, undo/redo sequences), and CppHighlighter (token recognition). (2) Integration tests for EditorWidget (simulate key sequences, verify buffer state). (3) Visual regression tests: screenshot comparison for rendering correctness. Use Qt Test framework (QTest) for all.

Q46. How would you implement a minimap (like VS Code's)?

A: Add a narrow QWidget on the right side of EditorWidget. In its paintEvent, render the entire document at ~1/10th scale using a tiny font (1-2px per character). Highlight the current viewport region with a semi-transparent overlay. Click/drag on the minimap sets scrollY proportionally. This requires rendering ALL lines (no culling) but at minimal resolution.

Q47. Explain the tradeoff between immediate-mode and retained-mode rendering for a text editor.

A: Valence uses immediate-mode: every frame, we redraw everything from scratch via paintEvent. Retained-mode would maintain a scene graph of rendered elements and only update changed parts. Immediate-mode is simpler (no cache invalidation bugs) but wastes work. Retained-mode is faster for large documents but complex to implement correctly. For our viewport-culled ~50-line renders, immediate-mode is the right choice.

Q48. How would you add git integration (gutter diff markers)?

A: Run 'git diff' via QProcess on file save. Parse the unified diff output to find changed/added/deleted line ranges. Store a vector<LineState> (unchanged/modified/added/deleted) and draw colored markers in the gutter during paintGutter(). Green for added, blue for modified, red triangle for deleted. Re-run diff on each save.

Q49. What's the hardest bug you encountered?

A: The infinite recursion crash. QAction shortcuts silently intercepted keyboard events and re-dispatched them, creating an invisible infinite loop. The editor would freeze for 1-2 seconds (as the stack grew) and then crash with no useful error message. Debugging required understanding Qt's event dispatch pipeline at a very deep level — specifically that QAction shortcuts fire before widget event handlers AND that sendEvent re-enters the same dispatch pipeline.

Part XIII — Key Code Walkthroughs

This section contains the actual implementation of the most interesting/complex functions with line-by-line explanations.

13.1 performUndo() — Reversing Edit Actions

This is one of the trickiest functions because undoing an Insert requires calculating the end position of the inserted text (to know what range to delete), and undoing a Delete requires re-inserting text at the original position.

```
void EditorWidget::performUndo() {
    auto result = undoManager_.undo();
    if (!result.valid) return;

    // Actions come in REVERSE order (last action first)
    for (auto& action : result.actions) {
        if (action.type == EditAction::Insert) {
            // Undo insert = delete the inserted text
            // Calculate end position by walking through the text
            Position end = action.pos;
            for (char c : action.text) {
                if (c == '\n') {
                    end.row++;
                    end.col = 0;
                } else {
                    end.col++;
                }
            }
            buffer_.deleteRange(action.pos, end);
            cursor_ = action.pos; // Cursor goes back to insertion point
        } else {
            // Undo delete = re-insert the deleted text
            Position newPos = buffer_.insertText(
                action.pos.row, action.pos.col, action.text);
            cursor_ = newPos; // Cursor goes to end of re-inserted text
        }
    }
    selection_.clear();
    setModified(true);
    updateGutterWidth();
    rebuildCommentState();
}
```

Key insight: for undoing an Insert of 'hello\nworld', we need to calculate that the end position is (startRow+1, 5). We walk through the text character by character, incrementing row on '\n' and col on everything else. Then deleteRange(start, end) removes exactly what was inserted.

13.2 paintCode() — Syntax-Highlighted Rendering

```
void EditorWidget::paintCode(QPainter& p, int startRow, int endRow) {
    p.setFont(font_);
    for (int row = startRow; row < endRow; row++) {
        int y = yFromRow(row);

        // Current line highlight (very subtle, alpha=6)
        if (row == cursor_.row)
            p.fillRect(gutterWidth_, y, width() - gutterWidth_,
                       charHeight_, Theme::CurrentLine);

        // Selection highlight
```



```

paintSelection(p, row, y);

// Syntax highlighting: tokenize and draw
bool inComment = blockCommentState_[row];
auto tokens = highlighter_.tokenize(buffer_.line(row), inComment);

for (const auto& tok : tokens) {
    QColor color;
    switch (tok.type) {
        case TokenType::Keyword:    color = Theme::SynKeyword; break;
        case TokenType::Type:       color = Theme::SynType; break;
        case TokenType::String:     color = Theme::SynString; break;
        case TokenType::Comment:    color = Theme::SynComment; break;
        case TokenType::Number:     color = Theme::SynNumber; break;
        case TokenType::Preprocessor: color = Theme::SynPreprocessor; break;
        case TokenType::Function:   color = Theme::SynFunction; break;
        case TokenType::Punctuation: color = Theme::SynPunctuation; break;
        default:                   color = Theme::TextPrimary; break;
    }
    p.setPen(color);
    int x = xFromCol(tok.start); // O(1) - monospace!
    p.drawText(x, y + ascent_,
              QString::fromStdString(
                  buffer_.line(row).substr(tok.start, tok.length)));
}
}
}

```

